

## Project structure code

```
backend/
├── main.py           # FastAPI entry point
├── config/
│   └── settings.py  # API keys, configs
├── routes/
│   ├── auth_routes.py
│   ├── rag_routes.py
│   └── upload_routes.py
├── services/
│   ├── rag_service.py  # RAG logic
│   ├── auth_service.py # JWT logic
│   └── db_service.py   # DB operations
├── models/
│   └── schemas.py     # request/response models
├── utils/
│   ├── data_loader.py
│   ├── embeddings.py
│   ├── vectorstore.py
│   └── search.py
└── database/
    └── connection.py

frontend/
└── app.py

data/
├── uploads/
└── faiss_index/
```

## Working logic of the code

---

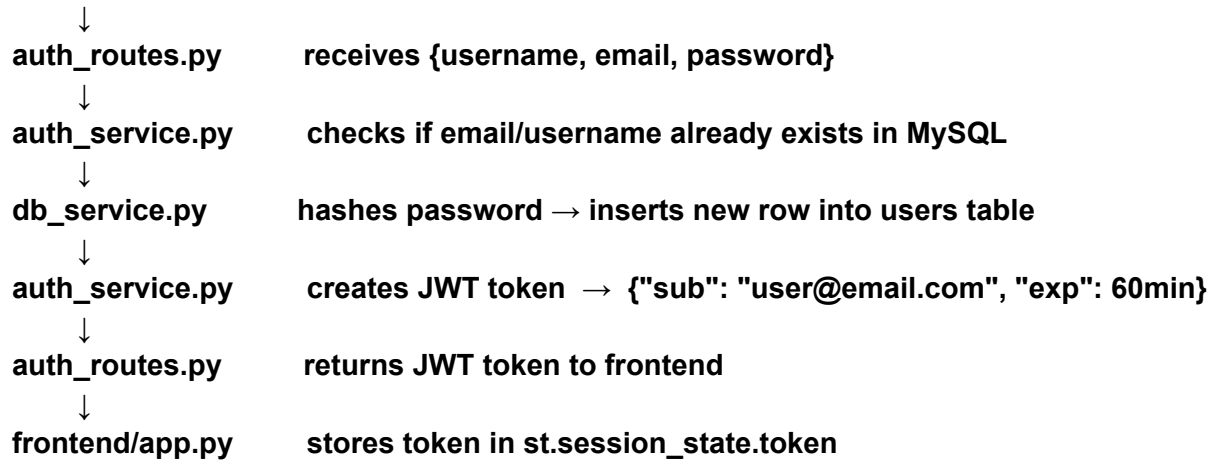
---

### SIGNUP / LOGIN

---

---

#### USER HITS SIGNUP



#### USER HITS LOGIN



---

---

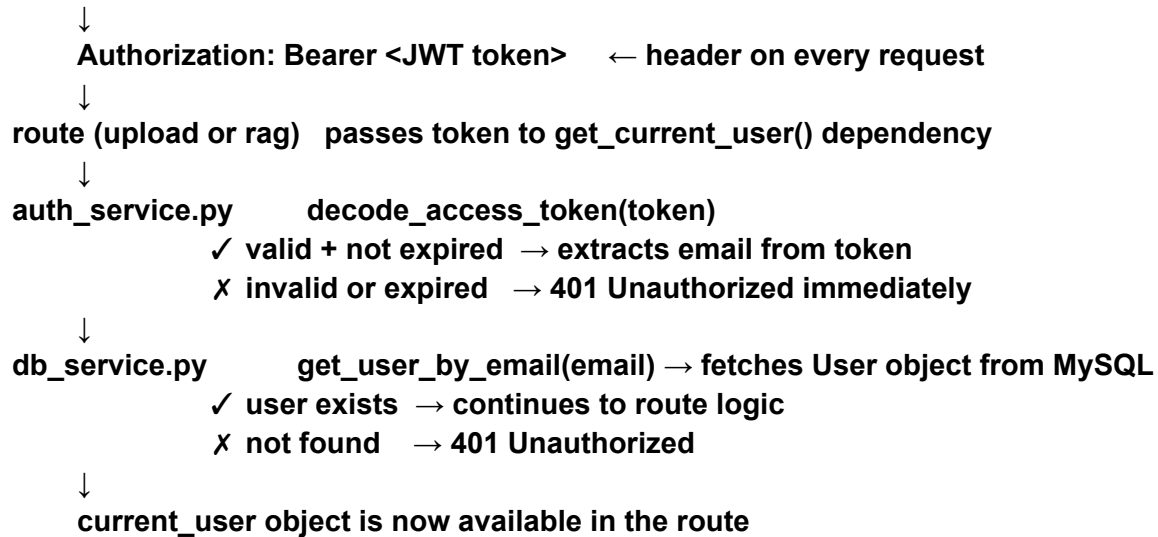
## EVERY REQUEST AFTER LOGIN

(JWT is verified on ALL routes)

---

---

### FRONTEND SENDS ANY REQUEST



---

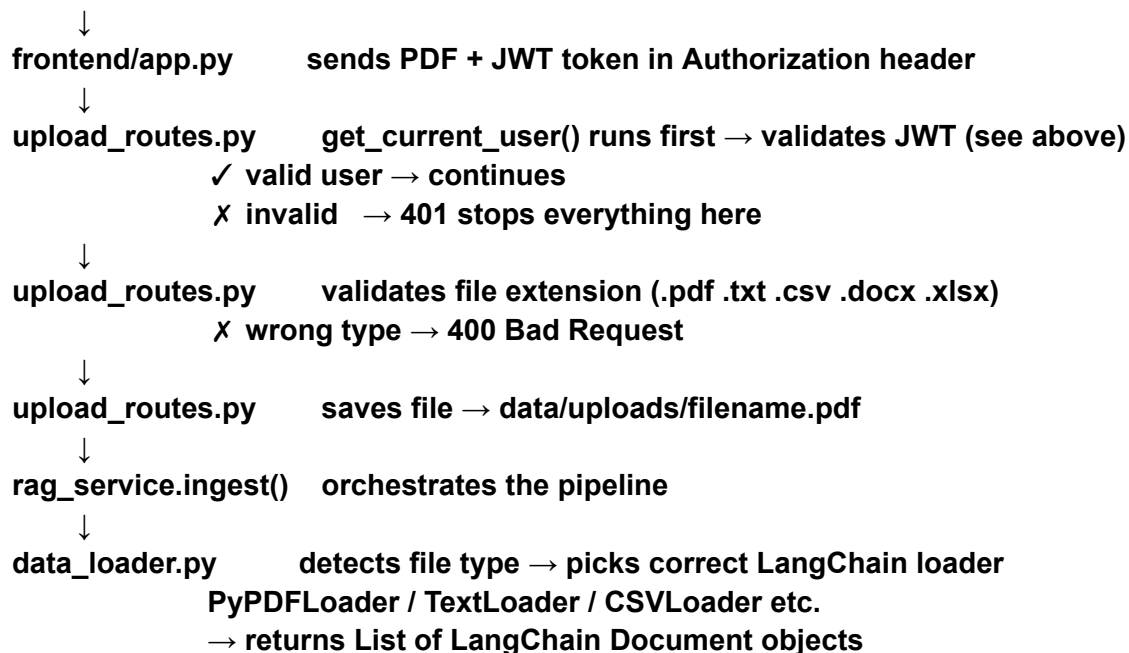
---

## FILE UPLOAD

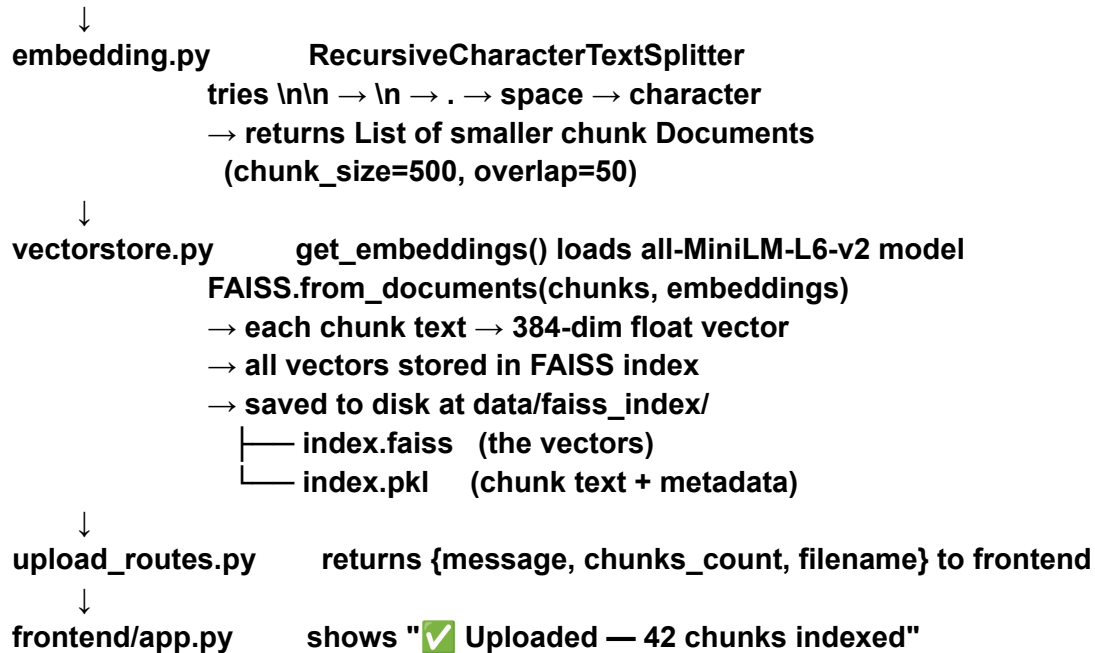
---

---

### USER UPLOADS A FILE



(one Document per page/section)



---

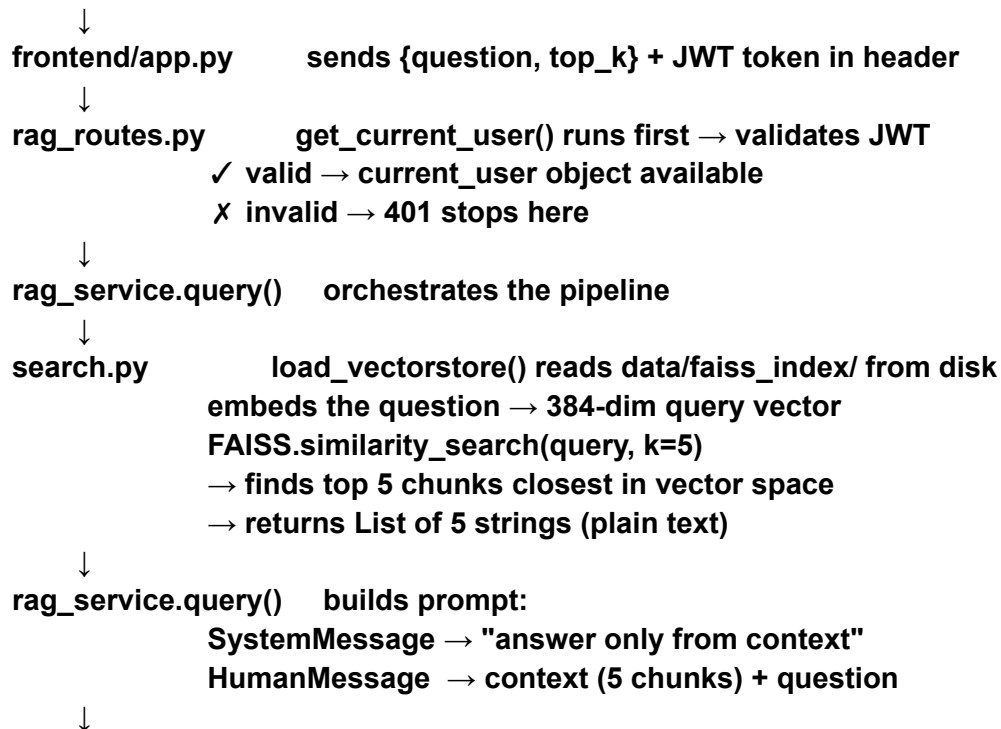
---

## USER ASKS A QUESTION

---

---

### USER TYPES A QUESTION



**GROQ Ai** reads context + question

→ generates answer strictly from context

X answer not in context →

"I could not find this in the document"



**db\_service.py** save\_chat(user\_id, question, answer, pdf\_name)

→ inserts row into MySQL chat\_history table

id | user\_id | question | answer | pdf\_name | created\_at



**rag\_routes.py** returns {answer, source\_chunks} to frontend



**frontend/app.py** displays answer in chat UI  
appends to st.session\_state.messages

---

---

## CHAT HISTORY

---

---

**USER CLICKS "LOAD HISTORY"**



**frontend/app.py** GET /rag/history + JWT token



**rag\_routes.py** validates JWT → gets current\_user



**db\_service.py** SELECT \* FROM chat\_history  
WHERE user\_id = current\_user.id  
ORDER BY created\_at DESC  
LIMIT 50



**rag\_routes.py** returns list of {question, answer, pdf\_name, created\_at}



**frontend/app.py** renders previous Q&A in chat UI

**USER CLICKS "CLEAR HISTORY"**



**frontend/app.py** DELETE /rag/history + JWT token



**rag\_routes.py** validates JWT → gets current\_user



**db\_service.py** DELETE FROM chat\_history WHERE user\_id = current\_user.id



**frontend/app.py** clears st.session\_state.messages

---

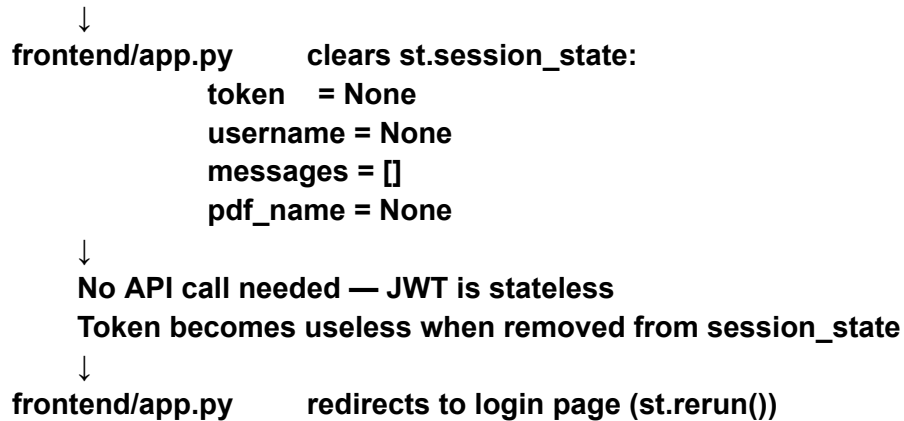
---

## LOGOUT

---

---

### USER CLICKS LOGOUT



### JWT Role During Login

When you log in, the JWT serves as the proof of a successful authentication.

- **Verification:** The server checks your credentials (like a username and password).
- **Token Generation:** If they are correct, the server generates a unique, digitally signed JWT.
- **Issuance:** This token is sent back to your browser or app, which usually stores it in local storage or a cookie.
- **Seamless Access:** From then on, the token is included in the header of every request you make, so the server knows you are still logged in without asking for your password again.

### Role During Signup

During signup, a JWT can be used to provide an immediate "logged-in" experience.

- **Account Creation:** You provide your details, and the server creates your account in the database.
- **Instant Session:** Instead of making you log in separately after registering, the server can immediately issue a JWT.
- **Onboarding:** This allows you to jump straight into using the application's protected features immediately after finishing the signup form.

# RAG BACKEND

## 1. Data\_loader logic:

Adding a single file path. If all directory files were taken then all of them would be chunked and vectorized again and again which would be a waste of time and resources. Right now the user can upload only one file at a time and there is no security for the duplicate files. Like if the file had already been uploaded and the user is uploading again the embeddings are made again and saved again no check for duplicacy. Will take care of that in later stages after the development of mvp.

## 2. Embeddings logic

Using hugging face sentence transformer embeddings they are fast and easy to use with no api key. Chunking is also done in this file itself with the help of the separate function.

## 3. Vectorstore

Used FAISS(Facebook AI Similarity Search)

Example::

YOUR PDF (3 pages)

Page 1: "Machine learning is..."

Page 2: "Neural networks are..."

Page 3: "Gradient descent works..."

↓ PyPDFLoader.load()

DOCUMENTS LIST (3 Document objects)

documents[0].page\_content = "Machine learning is..."

documents[0].metadata = {"source": "file.pdf", "page": 0}

documents[1].page\_content = "Neural networks are..."

documents[1].metadata = {"source": "file.pdf", "page": 1}

documents[2].page\_content = "Gradient descent works..."

documents[2].metadata = {"source": "file.pdf", "page": 2}

↓ RecursiveCharacterTextSplitter.split\_documents()

CHUNKS LIST (say 9 chunks — 3 per page)

chunks[0].page\_content = "Machine learning is a subset of..."  
chunks[1].page\_content = "...subset of AI that allows systems"  
chunks[2].page\_content = "...systems to learn from data"

chunks[3].page\_content = "Neural networks are inspired by..."  
chunks[4].page\_content = "...inspired by the human brain"  
chunks[5].page\_content = "...brain with layers of neurons"

chunks[6].page\_content = "Gradient descent works by..."  
chunks[7].page\_content = "...by minimizing a loss function"  
chunks[8].page\_content = "...function using derivatives"

↓ FAISS.from\_documents()

VECTORS IN FAISS (9 vectors, one per chunk)

---

---

Each chunk text → 384 numbers (embedding vector)  
FAISS stores all 384-dim vectors and searches them fast

The Document object is just a **wrapper** that keeps two things together:

Document

└─ page\_content → the actual text (this is what gets embedded)  
└─ metadata → where it came from (page number, file name)

For the vectorstore index.faiss stores the exact vectors and the index.pkl stores the {

```
vector_id: 25,  
text: "Worked on CNN architectures",  
  metadata: {  
    "source": "resume.pdf",  
    "page": 2  
  }  
}
```

Uses the vector building and vector loading for saving the vector embeddings and then loading the vectorstore at the time of retrieval for the similarity search

Settings.py - This file loads all the values from the env file which becomes easier in the code as only one object is initialized and used everywhere instead of doing os.getenv everywhere

4. **Search logic:** Loads the vectorstore and does a similarity check and provides the top k results for the query.



5. **Rag\_service:** Contains the data ingestion pipeline function and llm query answering function. The data ingestion function is called after the upload route uploads the file in the disk for creating and storing vectors in the vectorstore and the llm query answering function answers the query of the user by using the search function from the utils to search for the relevant chunks and create a structured prompt for the llm for answer the query accordingly.

## DATABASE

### SQLAlchemy Connection Pooling + RAG Backend Architecture Notes

#### 1. Core Understanding

`create_engine()` does NOT create one single DB connection

`</>` Python

```
engine = create_engine(  
    DATABASE_URL,  
    pool_pre_ping=True,  
    pool_size=5,  
    echo=False  
)
```



Run

The `engine` is:

Connection Manager + Connection Pool



Architecture:

```
FastAPI App  
↓  
SQLAlchemy Engine  
↓  
Connection Pool  
↓  
MySQL Database
```



## 2. What is Connection Pooling?

Instead of creating a new DB connection for every request:

```
Request →  
Open connection →  
Query →  
Close connection
```



SQLAlchemy keeps reusable connections ready.

Like:

```
Conn1  
Conn2  
Conn3  
Conn4  
Conn5
```



These are reused repeatedly.

## 3. pool\_size=5

Means:



```
Keep 5 reusable DB connections ready
```



## 4. What Happens When Many Requests Come?

Suppose:

```
pool_size=5
```



Requests:

```
Req1 → Conn1  
Req2 → Conn2  
Req3 → Conn3  
Req4 → Conn4  
Req5 → Conn5
```



All busy now.

Req6 waits briefly.

When Req1 finishes:

```
Conn1 becomes free
```

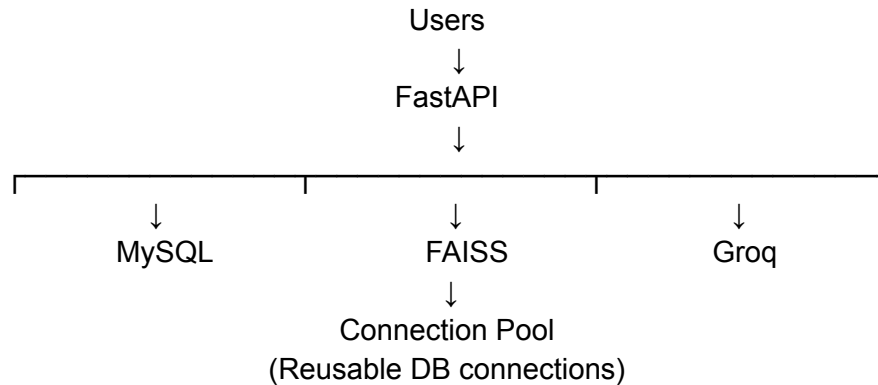


Then:

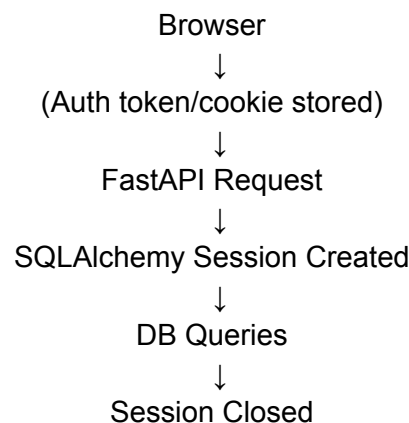
```
Req6 uses Conn1
```



This is connection reuse.



1. **Connection.py:** It uses sqlalchemy(ORM) to establish the connection between the database and the chatbot application.
2. **Schema.py:** It includes the validations for the routes or the api's we are going to use in the project along with the tables(user and chat) with their schema to be created automatically if already not present in the database.



## Browser/User Session (Authentication Session)

Browser/User Session is used to maintain the user's login/authentication state. When a user logs into an application, the server verifies the credentials and sends back a token or cookie (JWT token, session token, etc.). The browser stores this token and automatically sends it with future requests so the server knows the user is already authenticated. This session is usually long-lived and may last for hours or days. Multiple browser tabs on the same browser usually share the same authentication session because they share the same cookies/tokens. However, different devices like laptop and mobile have different authentication sessions because tokens are stored separately on each device.

## SQLAlchemy Session (Database Session)

SQLAlchemy Session is completely different from login/authentication session. It is a temporary backend database interaction workspace created during database operations. Usually, one HTTP request creates one SQLAlchemy session. This session is responsible for handling database queries, transactions, temporary object tracking, updates, inserts, deletes, and transaction isolation safely. The session internally borrows a database connection from the connection pool, performs required database operations, and then closes itself after the request finishes, returning the connection back to the pool.

The SQLAlchemy session temporarily stores/tracks retrieved database objects, pending changes, transaction state, and ORM mappings during the request lifecycle. For example, if a user object is fetched and modified, the session keeps track of those changes until `commit()` is called. Until the changes are committed, other sessions usually cannot see those updates. This isolation ensures safe and independent database operations between multiple users, requests, tabs, or devices.

## SERVICES

1. **Auth\_services.py:** It hashes the passwords securely, verify passwords during login, create JWT access tokens, decode JWT tokens and fetch the users from database
2. **Db\_services.py:** It consists of all the db functions involving creating user, saving the user chats, retrieving the user chats, deleting user chats

**NOTE:** Pdf is not the only file extension that could be uploaded in the application. Other file extensions are also allowed. For that to happen I have imported the `mimetypes` library which allows the correct identification of the file type by the frontend and the browser. For example if we upload the pdf file the frontend will use `application/pdf` which will tell the backend that it is a pdf file. Here `mimetypes` extracts the file types automatically. We don't need to add the file types manually.

**ROUTES-** Consists of all the route logics for the api's. The **FASTAPI** framework is used for creating these routes and the swagger api docs are created automatically which helps in the backend testing rather than using postman api.

1. **Auth\_route.py:**
2. **Rag\_route.py:**
3. **Upload\_route.py:**

# COMPLETE API STRUCTURE

## AUTH ROUTES

Prefix: /auth

Endpoints:

| Method | Route        | Purpose             |
|--------|--------------|---------------------|
| POST   | /auth/signup | Create new account  |
| POST   | /auth/login  | Login existing user |

## RAG ROUTES

Prefix: /rag

Endpoints:

| Method | Route        | Purpose               |
|--------|--------------|-----------------------|
| POST   | /rag/query   | Ask AI questions      |
| GET    | /rag/history | Get user chat history |

|        |              |                          |
|--------|--------------|--------------------------|
| DELETE | /rag/history | Delete user chat history |
|--------|--------------|--------------------------|

UPLOAD ROUTES

Prefix: /upload

Endpoints:

| Method | Route    | Purpose                     |
|--------|----------|-----------------------------|
| POST   | /upload/ | Upload and ingest documents |

# FRONTEND

